Lab Final

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

1 Lab Final

1. List the films whose rental rate is greater than the average rental rate of all films. Show film id, title, and rental rate.

```
SELECT film_id, title, rental_rate
FROM film
WHERE rental_rate > (
    SELECT AVG(rental_rate)
    FROM film
)
ORDER BY rental_rate DESC;
```

2. Find the film with the longest running time for each category and list their titles, category names, and lengths.

```
SELECT DISTINCT ON (c.category_id)
    c.name AS category_name,
    f.title,
    f.length
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
ORDER BY c.category_id, f.length DESC;
```

3. List all films that have titles consisting of more than one word and feature at least two different actors. Mention both the film and number of actors, sorted desc by number of actors.

```
SELECT f.title,
       COUNT(DISTINCT fa.actor_id) AS actor_count
FROM film f
JOIN film_actor fa ON f.film_id = fa.film_id
WHERE f.title LIKE '% %'
GROUP BY f.film_id, f.title
HAVING COUNT(DISTINCT fa.actor_id) >= 2
ORDER BY actor_count DESC, f.title;
```

4. For each category, show the category name and the total revenue collected from rentals of films in that category. Order by revenue descending.

```
SELECT c.name AS category_name,
       SUM(p.amount) AS total_revenue
FROM category c
JOIN film_category fc ON fc.category_id = c.category_id
JOIN inventory i ON i.film_id = fc.film_id
JOIN rental r ON r.inventory_id = i.inventory_id
JOIN payment p ON p.rental_id = r.rental_id
GROUP BY c.name
ORDER BY total_revenue DESC;
```

5. List customers who have rented films from at least 5 distinct categories. Show customer id, full name, and the number of distinct categories from which they rented.

```sql
SELECT c.customer_id, c.first_name, c.last_name,
       COUNT(DISTINCT fc.category_id) AS distinct_categories
FROM customer c
JOIN rental r ON r.customer_id = c.customer_id
JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film_category fc ON fc.film_id = i.film_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(DISTINCT fc.category_id) >= 5
ORDER BY distinct_categories DESC, c.customer_id;
```

6. Write an anonymous block that checks whether a given film title exists. If it exists, print its film_id; otherwise print `Film not found`.

```sql
DO $$
DECLARE
    v_title TEXT := 'Dracula Crystal';
    v_film_id INT;
BEGIN
    SELECT film_id
    INTO v_film_id
    FROM film
    WHERE title = v_title;

    IF FOUND THEN
        RAISE NOTICE 'Film found: % (film_id=%)', v_title, v_film_id;
    ELSE
        RAISE NOTICE 'Film not found: %', v_title;
    END IF;
END $$;
```

7. Write an anonymous block that sequentially prints the top 3 actors with the most film appearances, formatted as “Rank) First Name Last Name - X films”.

```sql
DO $$
DECLARE
    rec RECORD;
    i INT := 0;
BEGIN
    FOR rec IN
        SELECT a.actor_id, a.first_name, a.last_name, COUNT(*) AS film_count
        FROM actor a
        JOIN film_actor fa ON fa.actor_id = a.actor_id
        GROUP BY a.actor_id, a.first_name, a.last_name
        ORDER BY film_count DESC, a.actor_id
        LIMIT 3
    LOOP
        i := i + 1;
        RAISE NOTICE '%) % % - % films', i, rec.first_name, rec.last_name, rec.
    film_count;
    END LOOP;
END $$;
```

8. Create a function named get_customer_max_payment that takes a customer ID as an input and returns a message stating their highest payment amount. If the customer has no payment records, the function should return ”No payments found”.

```sql
CREATE OR REPLACE FUNCTION get_customer_max_payment(p_customer_id INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_max NUMERIC;
BEGIN
    SELECT MAX(amount)
    INTO v_max
    FROM payment
    WHERE customer_id = p_customer_id;

    IF v_max IS NULL THEN
        RETURN 'No payments found for customer ' || p_customer_id;
    ELSE
        RETURN 'Max payment for customer ' || p_customer_id || ' = ' || v_max;
    END IF;
END;
$$;

select * from get_customer_max_payment(1);
```

9. Create a procedure that deactivates customers by setting active = 0 for customers who have no rentals in the last p_days. Print how many customers were updated.

```sql
CREATE OR REPLACE PROCEDURE purge_inactive_customers(p_days INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_updated INT;
BEGIN
    -- Count the customers to deactivate
    SELECT COUNT(*) INTO v_updated
    FROM customer c
    WHERE c.active <> 0
      AND NOT EXISTS (
            SELECT *
            FROM rental r
            WHERE r.customer_id = c.customer_id
              AND r.rental_date >= NOW() - (p_days * INTERVAL '1 day')
      );

    -- Deactivate the customers
    UPDATE customer c
    SET active = 0
    WHERE c.active <> 0
      AND NOT EXISTS (
            SELECT *
            FROM rental r
            WHERE r.customer_id = c.customer_id
              AND r.rental_date >= NOW() - (p_days * INTERVAL '1 day')
      );

    RAISE NOTICE 'Customers deactivated: % (threshold=% days)', v_updated,
    p_days;
END;
$$;
```

```
CALL purge_inactive_customers(30);
```

10. Write a function named relocate_customers that updates the city for all customers currently living in a specific city. The function should take the "old city" and "new city" as input parameters and return the total number of customers who were moved.

```
CREATE OR REPLACE FUNCTION relocate_customers(p_old_city VARCHAR, p_new_city
    VARCHAR)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_old_city_id INT;
    v_new_city_id INT;
    v_moved_count INT;
BEGIN
    -- Resolve string names to numeric primary keys
    SELECT city_id INTO v_old_city_id FROM city WHERE city = p_old_city;
    SELECT city_id INTO v_new_city_id FROM city WHERE city = p_new_city;

    -- Exit early if either city does not exist in the database
    IF v_old_city_id IS NULL OR v_new_city_id IS NULL THEN
        RETURN 0;
    END IF;

    -- Step 1: Pre-calculate the total number of customers residing in the old
    city
    SELECT COUNT(c.customer_id) INTO v_moved_count
    FROM customer c
    JOIN address a ON c.address_id = a.address_id
    WHERE a.city_id = v_old_city_id;

    -- Step 2: Update the city_id for all addresses currently associated with
    those customers
    UPDATE address
    SET city_id = v_new_city_id
    WHERE city_id = v_old_city_id
      AND address_id IN (SELECT address_id FROM customer);

    -- Step 3: Return the integer count
    RETURN v_moved_count;
END;
$$;
SELECT relocate_customers('London', 'Manchester');
```

2 ER Diagram


actor
actor_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
last_update DATETIME *
language
language_id INT * pk
name CHAR(20) *
last_update DATETIME *
film
film_id INT * pk
title VARCHAR(255) *
description TEXT
release_year YEAR
language_id INT *
rental_duration SMALLINT *
rental_rate DECIMAL(4,2) *
length SMALLINT
replacement_cost DECIMAL(5,2) *
rating film_rating_enum
last_update DATETIME *
special_features TEXT
fulltext_col TEXT *
category
category_id INT * pk
name VARCHAR(25) *
last_update DATETIME *
country
country_id INT * pk
country VARCHAR(50) *
last_update DATETIME *
city
city_id INT * pk
city VARCHAR(50) *
country_id INT *
last_update DATETIME *
address
address_id INT * pk
address VARCHAR(50) *
address2 VARCHAR(50)
district VARCHAR(20) *
city_id INT *
postal_code VARCHAR(10)
phone VARCHAR(20) *
last_update DATETIME *
film_actor
actor_id INT *
film_id INT *
last_update DATETIME *
inventory
inventory_id INT * pk
film_id INT *
store_id INT *
last_update DATETIME *
film_category
film_id INT *
category_id INT *
last_update DATETIME *
customer
customer_id INT * pk
store_id INT *
first_name VARCHAR(45) *
last_name VARCHAR(45) *
email VARCHAR(50)
address_id INT *
activebool BOOLEAN *
create_date DATE *
last_update DATETIME
active INT
staff
staff_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
address_id INT *
email VARCHAR(50)
store_id INT *
active BOOLEAN *
username VARCHAR(16) *
password VARCHAR(40)
last_update DATETIME *
picture BLOB
rental
rental_id INT * pk
rental_date DATETIME *
inventory_id INT *
customer_id INT *
return_date DATETIME
staff_id INT *
last_update DATETIME *
store
store_id INT * pk
manager_staff_id INT *
address_id INT *
last_update DATETIME *
payment
payment_id INT * pk
customer_id INT *
staff_id INT *
rental_id INT
amount DECIMAL(5,2) *
payment_date DATETIME *